Self-Healing Multi-Agent Systems: Autonomous Failure Detection, Root-Cause Analysis, and Adaptive Plan Repair for Enterprise AI Workflows

Dambara Naidoo[1], Priya Krishnamurthy[2], James O. Hartwell[1]

[1]Department of Computer Science and Artificial Intelligence, University of Technology, Pretoria, South Africa

[2]Enterprise AI Research Laboratory, Institute for Advanced Computing, Johannesburg, South Africa

Corresponding Author: d.naidoo@utpretoria.ac.za | Received: July 2026 | DOI: 10.1109/TNNLS.2026.XXXXXXX

Abstract Autonomous AI agent systems deployed in enterprise environments frequently encounter runtime failures arising from network instability, data schema violations, resource exhaustion, and tool incompatibilities. Existing multi-agent frameworks lack systematic mechanisms for autonomous recovery, leading to cascading failures, unhandled escalations, and complete cessation of workflow execution. This paper presents **SH-MAS** (Self-Healing Multi-Agent System), a production-grade framework equipping agents with four tightly integrated capabilities: (1) a rule-based failure taxonomy classifying runtime errors across six canonical categories with sub-100 ms latency; (2) a reflection-augmented root-cause analysis (RCA) engine combining LLM reasoning with a dynamically updated knowledge graph; (3) a six-strategy repair engine applying targeted plan mutations without restarting the workflow; and (4) a dual-store memory system (episodic + semantic) with reinforcement-based strategy scoring driving continuous improvement. Experimental evaluation across three failure scenarios demonstrates a mean **Repair Rate (RR) of 0.847**, **MTTR of 340 ms**, **Plan Success Rate (PSR) of 0.912**, and **Learning Performance Index (LPI) of +0.143**, establishing SH-MAS as a significant advance over static multi-agent architectures.

Keywords: Multi-agent systems, self-healing systems, fault tolerance, autonomous repair, LLM-based reasoning, knowledge graphs, episodic memory, LangGraph, enterprise AI, failure classification.

1. Introduction

The deployment of autonomous agent systems in enterprise settings has accelerated dramatically with the maturation of large language models (LLMs) and agentic frameworks [1]. Modern workflows increasingly delegate complex, multi-step tasks—data retrieval, cross-system integration, report generation, notification dispatch—to orchestrated agent pipelines that interact with external APIs, relational databases, file systems, and third-party services. These environments are inherently non-deterministic and subject to transient and persistent failures.

Despite this operational reality, the vast majority of production multi-agent systems are architecturally brittle: they execute predefined plans, detect failure via simple exception propagation, and either halt execution or retry blindly without reasoning about the root cause. This creates three fundamental problems:

- **Failure Opacity.** Exceptions carry syntactic information but no semantic classification that guides repair decisions.
- **Static Recovery.** Fixed retry policies ignore causal failure structure — retrying a network timeout is sensible; retrying a schema validation error with identical parameters is not.
- **Amnesiac Behaviour.** Each workflow execution is independent. Knowledge about effective repair strategies is discarded after every run.

These limitations motivate a fundamentally different paradigm: *self-healing* agent systems that autonomously detect, diagnose, repair, and learn from failures as first-class workflow lifecycle operations.

1.1 Contributions

This paper makes five primary contributions:

- **SH-MAS Framework.** The first complete open-source framework for self-healing enterprise multi-agent workflows, integrating failure taxonomy, LLM-driven RCA, structured repair strategy selection, and continuous memory-based learning within a single coherent graph execution model.
- **Formal Metrics.** Definition of five evaluation metrics — MTTR, RR, PSR, FDR, and LPI — with closed-form formulae and validated measurability.
- **Hybrid Classifier.** Demonstration that combining a rule-based classifier with LLM reflection and knowledge-graph augmentation outperforms either approach alone, maintaining sub-100 ms classification latency.
- **Learning Evidence.** Statistical evidence of memory-driven learning across repeated executions (LPI = +0.143, $p < 0.01$).
- **MCP Integration.** First demonstration of MCP protocol integration in a self-healing agent framework, enabling invocation from Claude Desktop, VS Code Copilot, and compatible clients.

1.2 Paper Organisation

Section 2 reviews related work. Section 3 presents system architecture. Section 4 details components. Section 5 describes experimental methodology. Section 6 reports results. Section 7 discusses implications and limitations. Section 8 concludes.

2. Related Work

2.1 Multi-Agent Architectures

The multi-agent systems (MAS) literature spans several decades [2, 3]. BDI (Belief-Desire-Intention) architectures [4] established reactive and deliberative reasoning but did not address continuous learning from failures in LLM pipelines. Contemporary frameworks — LangChain [5], AutoGen [6], CrewAI [7], and LangGraph [8] — provide orchestration primitives but treat failure handling as an application-level concern. ReAct [9] introduced reasoning-action interleaving, significantly improving single-step task reliability, but lacks stateful repair histories or structured failure classification.

2.2 Fault Tolerance and Self-Healing

Self-healing in distributed systems [10, 11] employs watchdog processes, circuit breakers (Hystrix [12]), and restart policies operating at the infrastructure layer without semantic failure reasoning. Chaos engineering [13] and Kubernetes resilience [14] test resilience rather than achieving autonomous repair. IBM's Autonomic Computing vision [15] articulated self-healing as a self-* property but implementations were rule-based and domain-specific.

2.3 LLM-Assisted Debugging and Repair

Recent work explores LLMs for bug explanation [16], patch generation [17], and SWE-Bench evaluation [18, 19]. These approaches target offline code repair. SH-MAS differs by targeting runtime workflow failures in live executions with sub-second latency requirements. Reflexion [20] introduced verbal reinforcement learning via episodic memory; SH-MAS extends this with structured failure taxonomy, semantic strategy scoring, and knowledge graph augmentation over multi-step plans.

2.4 Knowledge Graphs in Agent Systems

Knowledge graphs augment reasoning in QA [21], recommendation [22], and scientific discovery [23]. Zhu et

al. [24] use KGs for inter-task dependency storage; Park et al. [25] for generative agent memory. SH-MAS uses a directed KG with weighted tool → failure-type → repair-strategy edges updated after every execution, enabling predictive strategy selection from historical evidence.

2.5 Positioning

SH-MAS uniquely intersects LLM semantic reasoning (Reflexion, ReAct), structured failure taxonomy (autonomic computing), stateful graph workflows (LangGraph), and continuously updated knowledge graphs with dual-store memory. No existing system combines all four capabilities in a production-deployable package.

3. System Architecture

3.1 Design Principles

SH-MAS is designed around four principles:

- **Separation of Concerns.** Each capability (planning, execution, detection, repair, learning) is a distinct graph node with interface: state_in → state_out.
- **Immutable State Updates.** No node mutates state in place; each returns a partial dictionary of changed keys, preserving auditability and enabling replay.
- **Stateless Services.** FailureDetector, PlanRepairer, and LearningEngine are stateless — behaviour is determined entirely by state and external memory/KG stores.
- **Graceful Degradation.** Every repair strategy has a fallback (ultimately ESCALATE), preventing infinite loops or silent failures.

3.2 Graph Topology

The SH-MAS workflow is a directed conditional graph implemented using LangGraph's StateGraph (Figure 1). The main path is: START → planner → executor → learner → finalizer → END. On failure, execution routes through: failure_detector → root_cause_analyzer → plan_repairer. The repairer routes back to executor (retry strategies) or forward to finalizer (ESCALATE). The healing loop executes up to max_repairs times before forced termination.

3.3 State Model

The AgentState typed dictionary serves as single source of truth for all nodes:

Field	Type	Description
task_id	str	UUID for current execution
objective	str	Natural language task description
plan	list[PlanStep]	Ordered execution steps
current_step_index	int	Pointer to active step
step_results	list[StepResult]	Per-step execution history
failures	list[Failure]	Failures with RCA annotations
repair_count	int	Repair cycles consumed
max_repairs	int	Upper bound on repairs
status	AgentStatus	Current lifecycle status
metrics	HealingMetrics	Aggregated performance data

Table A. AgentState key fields.

4. Component Design

4.1 Failure Taxonomy

The FailureTaxonomy module (core/knowledge/taxonomy.py) provides the semantic bridge between raw exception text and structured repair decisions. Six canonical failure types are defined with regex-based detection:

Type	Severity	Example Patterns
NETWORK	HIGH	connection refused, timed out, ssl error
TOOL	MEDIUM	attribute error, permission denied
DATA	MEDIUM	validation error, json decode, field required
RESOURCE	CRITICAL	429, rate limit, out of memory
DEPENDENCY	HIGH	prerequisite, not yet available
LOGIC	MEDIUM	assertion error, unexpected output

Table B. Failure taxonomy classification rules.

Classification uses compiled regular expressions evaluated against lowercased error strings, achieving O(n×m) complexity (n rules, m patterns) with sub-millisecond throughput. The taxonomy is seeded at startup and extensible without modifying core logic.

4.2 Failure Detection Node

The FailureDetector (core/healing/detector.py) is a stateless classifier invoked immediately after executor failure. It enriches the raw Failure dict with failure_type and severity by delegating to classify_error(). The design is deliberately lightweight — no LLM call — to minimise repair latency. Classification completes in under 1 ms for typical error strings.

4.3 Reflection-Augmented RCA

The root_cause_analyzer_node implements a three-phase pipeline:

- **Phase 1 — KG Augmentation.** Queries the KnowledgeGraph for historically effective strategies for the current (failure_type, tool) pair.
- **Phase 2 — LLM Reflection.** Constructs a structured prompt with: failing step context, classified failure type, last-5 step history, and KG-retrieved patterns. Returns a Pydantic-validated _RCAOutput with root_cause, confidence (0–1), repair_strategy, repair_rationale, modified_parameters, and fallback_tool.
- **Phase 3 — State Update.** Enriches the Failure record with RCA fields and forwards the full response to plan_repairer.

LLM temperature is set to 0.1 (near-deterministic) to maximise consistency of strategy selection across similar failure patterns. The Groq llama-3.3-70b-versatile model completes RCA in 300–400 ms.

4.4 Six-Strategy Repair Engine

The RepairAgent (core/agents/repair.py) implements a strategy dispatch table. Each strategy is a pure async method returning a partial state update, preserving immutability:

Strategy	Condition	Action
RETRY	retry_count < max_retries	Re-execute step unchanged
RETRY_MODIFIED	RCA provides modified_parameters	Re-execute with LLM adjustments
FALLBACK	fallback_tool available	Swap tool, reset retry counter
REPLAN	Multiple steps affected	LLM regenerates plan from idx
SKIP	is_optional == True	Advance index, mark skipped
ESCALATE	Max retries exhausted	Mark ABORTED, surface to human

Table C. Six-strategy repair engine.

4.5 Dual-Store Memory System

Episodic Memory (core/memory/episodic.py) is an append-only SQLite log of complete task executions. Each Episode stores the full plan, step results, failures with RCA annotations, and metrics as JSON-serialised fields. Retrieval uses keyword overlap scoring between the query string and stored objectives.

Semantic Memory (core/memory/semantic.py) stores scored strategy entries. Scores are updated through reinforcement (success → score × (1 + lr)) and decay (failure → score × (1 – lr)) where lr = 0.15. Top-k=5 entries are injected into the planner prompt as learned_context.

4.6 Knowledge Graph

The KnowledgeGraph (core/knowledge/graph.py) is a directed NetworkX DiGraph with node types: tool, failure_type, strategy. Edge types include: causes (tool → failure_type), repaired_by (failure_type → strategy with success_count/failure_count), failed_with, and succeeds_via. The graph is seeded with baseline patterns and updated by LearningEngine after every execution, continuously refining predictive accuracy. Persisted as pickle (data/knowledge_graph.pkl); Neo4j adapter is straightforward given the clean interface.

4.7 LLM Provider Abstraction

LLMFactory supports five provider backends with @lru_cache(maxsize=4) for singleton caching:

Provider	Models	Use Case
OpenAI	gpt-4o, gpt-4o-mini	Cloud production
Anthropic	claude-3-5-sonnet	High-accuracy RCA
Azure OpenAI	Deployment-specific	Enterprise data residency
Groq	llama-3.3-70b-versatile	Low-latency throughput
Ollama	llama3.2, mistral	Air-gapped / on-premises

Table D. Supported LLM provider backends.

4.8 Tool Ecosystem and MCP Integration

SH-MAS ships with six enterprise tools (WebSearchTool, DatabaseQueryTool, APIClientTool, FileProcessorTool, NotifierTool, NoOpTool) each exposing a failure_rate attribute for controlled fault injection during experiments. The ToolRegistry maintains a singleton name→tool map. tools/mcp/server.py wraps the entire framework as an MCP 0.1 server, enabling direct invocation from Claude Desktop, VS Code Copilot, or any MCP-compatible client [26, 27].

4.9 REST API and Observability

The FastAPI service layer exposes: POST /tasks/ (submit task), GET /tasks/{id} (retrieve result), GET /health (liveness), GET /healing/stats (aggregate metrics), GET /knowledge/stats (KG stats). Structured logging via structlog emits JSON events for every workflow transition, integrating with ELK, Datadog, or Azure Monitor. A React/TypeScript dashboard (Vite + TailwindCSS) provides real-time task monitoring and knowledge graph inspection.

5. Experimental Methodology

5.1 Evaluation Metrics

We define five metrics to quantitatively evaluate SH-MAS:

Repair Rate (RR): RR = |{f ∈ F : f.resolved = True}| / |F|, where F is all failures in a scenario run.

Mean Time to Repair (MTTR): MTTR = Σ t_repair_i / n, wall-clock duration of each repair cycle (ms).

Plan Success Rate (PSR): PSR = |{r ∈ R : r.status = completed}| / |R|, fraction of ru escalation.

Failure Detection Rate (FDR): FD / all failures, precision against h set.

Learning Performance Index (LPI): LPI = RR_late − RR_early. Positive LPI confirms learning over time.

5.2 Experimental Scenarios

Scenario A — Network Degradation (30%): APIClientTool and WebSearchTool configured with failure_rate = 0.30, simulating 30% of calls failing with ConnectionError. Tests RETRY and FALLBACK strategies.

Scenario B — Database Failure (40%): DatabaseQueryTool with failure RETRY, RETRY_MODIFIED, and

Scenario C — Cascading Mul enterprise tools with failure_rate The hardest scenario; tests REI selection under compound failure

Each scenario executes N = 10 runs with 5 rotating enterprise objectives (data retrieval, report generation, notification dispatch, configuration validation, customer analytics). max_repairs = 3.

5.3 Baselines

- **B1 — No Repair:** Workflow halts on first failure; no recovery mechanism.
- **B2 — Fixed Retry:** Three retries with identical parameters; no semantic classi
- **B3 — Random Strategy:** Repair strategy selected uniformly at random from six options.

5.4 Implementation Details

Environment: Windows 11, Intel Core i7, 16 GB RAM, Python 3.14, LangGraph 0.4, LangChain Core 0.3, Groq llama-3.3-70b-versatile (temperature=0.1), SQLite, NetworkX 3.x, asyncio. Tool failures use random.random() < failure_rate with fixed seed for reproducibility.

6. Results

6.1 Repair Rate and Plan Success Rate

Table 1 presents the primary performance metrics. SH-MAS achieves a mean PSR of 0.910, representing a 72.9 percentage-point improvement over the No-Repair baseline and 38.3 pp over Fixed Retry. The gap is largest in Scenario C, confirming that semantic strategy selection is critical under compound failure conditions.

System	Scen A PSR/RR	Scen B PSR/RR	Scen C PSR/RR	Mean P
B1 — No Repair	0.12 / 0.00	0.08 / 0.00	0.05 / 0.00	0.083 /
B2 — Fixed Retry	0.65 / 0.51	0.52 / 0.44	0.41 / 0.38	0.527 /
B3 — Random	0.70 / 0.57	0.60 / 0.52	0.48 / 0.45	0.593 /
SH-MAS (ours)	0.94 / 0.89	0.91 / 0.84	0.88 / 0.81	0.910 /

Table 1. Comparative performance across failure scenarios (N=10 runs each). Bold = SH-MAS best.

6.2 Mean Time to Repair

SH-MAS achieves a mean MTTR of 340 ms — approximately 7.3× faster than Fixed Retry (2,480 ms). This is attributable to the rule-based classifier completing in < 1 ms and the LLM RCA call completing in 300–400 ms on Groq. Baselines accumulate delays by exhausting retry budgets before escalating.

Scenario	B2 Fixed Retry	B3 Random	SH-MAS
A — Network 30%	1,820 ms	2,340 ms	342 ms
B — Database 40%	2,150 ms	2,890 ms	358 ms
C — Cascade 20%	3,470 ms	4,120 ms	320 ms
Mean	2,480 ms	3,117 ms	340 ms

Table 2. Mean Time to Repair (MTTR) in milliseconds.

6.3 Failure Detection Rate

Against a hand-labelled reference set of 150 synthetic failures (25 per type), the taxonomy classifier achieves a macro-average F1 of 0.877. RESOURCE failures have highest F1 (0.97) due to distinctive HTTP status codes. LOGIC failures have lowest F1 (0.77) as they manifest in output content rather than exception messages.

Failure Type	Precision	Recall	F1
NETWORK	0.96	0.92	0.94

pe	Precision	Recall	F1
	0.88	0.84	0.858
	0.91	0.88	0.895
	0.98	0.96	0.969
	0.85	0.80	0.824
	0.79	0.76	0.77
	0.895	0.860	0.877

Table 3. Per-class Failure Detection Rate against 150 labelled samples.

6.4 Learning Performance Index

All three scenarios yield positive LPI (Table 4), confirming that SH-MAS improves repair effectiveness as episodic memory accumulates history and knowledge graph edges converge. The largest gain is in Scenario C (LPI = +0.160), consistent with cascading failures providing the richest diversity of learning signals.

io	Early RR	Late RR	LPI
	0.82	0.95	+0.130
	0.80	0.93	+0.130
	0.74	0.90	+0.160
	0.787	0.927	+0.143

Table 4. Learning Performance Index across scenarios.

6.5 Strategy Selection Distribution

gy	Frequency	Success Rate
	38.2%	72.4%
FIED	22.7%	88.1%
	19.5%	91.3%
	14.1%	83.7%
	4.3%	100%
	1.2%	N/A

Table 5. Repair strategy frequency and success rate across all scenarios.

RETRY_MODIFIED and FALLBACK achieve the highest success rates (88.1% and 91.3%), validating the importance of semantic parameter adjustment over blind repetition. ESCALATE is triggered in only 1.2% of repair attempts, demonstrating effective recovery before human escalation.

7. Discussion

7.1 Ablation: Taxonomy + LLM vs. Either Alone

Table 6 confirms that combining rule-based taxonomy with LLM reflection outperforms either alone. Taxonomy-only is fast (215 ms) but coarse. LLM-only is slower (1,240 ms) and less consistent without structured classification input. The combined approach achieves the best PSR (0.91) and RR (0.84) at an acceptable MTTR of 358 ms.

Configuration	PSR	RR	MTTR
Taxonomy only (no LLM RCA)	0.74	0.71	215 ms
LLM RCA only (no taxonomy)	0.83	0.78	1,240 ms
Taxonomy + LLM RCA (SH-MAS)	0.91	0.84	358 ms

Table 6. Ablation study — Scenario B (Database Failure, N=10).

7.2 Memory System Contribution

Running Scenario A with memory disabled (no episodic/semantic recall during planning) reduces LPI to +0.031, near-zero. This confirms that learning gains are specifically attributable to memory-augmented planning, not incidental effects of repeated execution.

7.3 Limitations

- **LOGIC Classification.** F1 = 0.77 is the lowest class. Future work will augment LOGIC detection with a lightweight LLM output classifier.
- **KG Scalability.** NetworkX + pickle is suitable for prototypes but not deployments exceeding 10,000 nodes. Neo4j or Amazon Neptune adaptation is straightforward given the clean KnowledgeGraph interface.
- **LLM Non-Determinism.** At temperature=0.1, outputs are nearly but not fully deterministic. A voting mechanism (3 LLM calls, majority selection) would increase consistency.
- **Evaluation Scope.** Experiments use simulated tools with stochastic failure injection. Evaluation against production APIs and databases is planned as future work.

7.4 Industrial Applicability

SH-MAS addresses three enterprise pain points: (1) on-call alert fatigue from trivially recoverable failures; (2) SLA breaches from unnecessarily halted workflows; (3) inability of agent systems to improve from operational experience [28, 29, 30]. The REST API and MCP integration enable adoption as a drop-in orchestration layer in front of existing tool ecosystems.

8. Conclusion

This paper presented SH-MAS, a self-healing multi-agent framework for enterprise AI workflows. The combination of typed failure taxonomy, reflection-augmented RCA, a six-strategy repair engine, and a dual-store memory system with knowledge graph backing achieves: mean PSR = 0.910, RR = 0.847, MTTR = 340 ms, and LPI = +0.143 across three controlled failure scenarios. These results establish SH-MAS as a significant advance over static architectures and reactive monitoring approaches, confirming that autonomous failure reasoning and

memory-based learning are essential capabilities for robust enterprise AI deployment.

The framework is open-source, production-deployable via FastAPI/Docker, MCP-compatible, and provider-agnostic across five LLM backends. Future work will address LOGIC failure semantic detection, graph database backends, and evaluation in live enterprise environments.

Acknowledgements

The authors gratefully acknowledge the open-source communities behind LangGraph, LangChain, FastAPI, NetworkX, and ReportLab, whose libraries underpin this work. We thank the reviewers for their constructive and detailed feedback that strengthened the paper.

References

[1] Wang, L., Ma, C., Feng, X. et al. (2024). "A Survey on Large Language Model based Autonomous Agents." *Frontiers of Computer Science*, 18(6), 186345. https://doi.org/10.1007/s11704-024-40231-1

[2] Wooldridge, M. and Jennings, N. R. (1995). "Intelligent Agents: Theory and Practice." *The Knowledge Engineering Review*, 10(2), 115–152. https://doi.org/10.1017/S0269888900008122

[3] Ferber, J. (1999). *Multi-Agent Systems: An Introduction to Distributed Artificial Intelligence*. Addison-Wesley Longman. ISBN 978-0-201-36048-6.

[4] Rao, A. S. and Georgeff, M. P. (1995). "BDI Agents: From Theory to Practice." *Proc. ICMAS-95*, San Francisco, pp. 312–319.

[5] Chase, H. (2022). "LangChain: Building Applications with LLMs through Composability." GitHub. https://github.com/langchain-ai/langchain [Accessed: July 2026].

[6] Wu, Q., Bansal, G., Zhang, J. et al. (2023). "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation." *arXiv:2308.08155*. https://arxiv.org/abs/2308.08155

[7] Moura, J. G. and Carraro, J. P. (2024). "CrewAI: Framework for Orchestrating Role-Playing Autonomous AI Agents." GitHub. https://github.com/crewAIInc/crewAI

[8] LangChain Inc. (2024). "LangGraph: Stateful Multi-Actor Applications with LLMs." https://langchain-ai.github.io/langgraph/

[9] Yao, S., Zhao, J., Yu, D. et al. (2023). "ReAct: Synergizing Reasoning and Acting in Language Models." *ICLR 2023*. https://arxiv.org/abs/2210.03629

[10] Psaier, H. and Dustdar, S. (2011). "A Survey on Self-Healing Systems." *Computing*, 91(1), 43–73. https://doi.org/10.1007/s00607-010-0107-y

[11] Kephart, J. O. and Chess, D. M. (2003). "The Vision of Autonomic Computing." *IEEE Computer*, 36(1), 41–50. https://doi.org/10.1109/MC.2003.1160055

[12] Netflix Technology Blog (2012). "Introducing Hystrix for Resilience Engineering." https://netflixtechblog.com/introducing-hystrix-for-resilience-engineering-13531c1ab362

[13] Basiri, A., Behnam, N., de Rooij, R. et al. (2016). "Chaos Engineering." *IEEE Software*, 33(3), 35–41. https://doi.org/10.1109/MS.2016.60

[14] Burns, B., Grant, B., Oppenheimer, D. et al. (2016). "Borg, Omega, and Kubernetes." *ACM Queue*, 14(1), 70–93. https://doi.org/10.1145/2898442.2898444

[15] IBM Corporation (2006). *An Architectural Blueprint for Autonomic Computing*, 4th ed. IBM White Paper.

[16] Sobania, D., Briesch, M., Hanna, C. and Petke, J. (2023). "Analysis of ChatGPT Automatic Bug Fixing." *Proc. APR 2023*, pp. 23–30. https://arxiv.org/abs/2301.08653

[17] Xia, C. S. and Zhang, L. (2023). "Keep the Conversation Going: Fixing 162 of 337 Bugs for $0.42 each." *arXiv:2304.00385*.

[18] Jimenez, C. E., Yang, J., Wettig, A. et al. (2024). "SWE-bench: Can LMs Resolve Real-World GitHub Issues?" *ICLR 2024*. https://arxiv.org/abs/2310.06770

[19] Bader, J., Scott, A., Pradel, M. and Chandra, S. (2019). "Getafix: Learning to Fix Bugs Automatically." *PACMPL*, 3(OOPSLA), Art. 159. https://doi.org/10.1145/3360585

[20] Shinn, N., Cassano, F., Labash, B. et al. (2023). "Reflexion: Language Agents with Verbal Reinforcement Learning." *NeurIPS 2023*, 36, 8634–8652. https://arxiv.org/abs/2303.11366

[21] Ji, S., Pan, S., Cambria, E. et al. (2022). "A Survey on Knowledge Graphs." *IEEE TNNLS*, 33(2), 494–514. https://doi.org/10.1109/TNNLS.2021.3070843

[22] Wang, H., Zhang, F., Hou, M. et al. (2018). "SHINE: Signed Heterogeneous Information Network Embedding." *Proc. WSDM 2018*, pp. 592–600. https://doi.org/10.1145/3159652.3159666

[23] Nicholson, D. N. and Greene, C. S. (2020). "Constructing Knowledge Graphs and Their Biomedical Applications." *Comp. Struct. Biotechnology Journal*, 18, 1414–1428. https://doi.org/10.1016/j.csbj.2020.05.017

[24] Zhu, X., Chen, W., Tian, H. et al. (2023). "Ghost in the Minecraft: Generally Capable Agents for Open-World Environments." *arXiv:2305.17144*.

[25] Park, J. S., O'Brien, J. C., Cai, C. J. et al. (2023). "Generative Agents: Interactive Simulacra of Human Behavior." *Proc. UIST 2023*, pp. 1–22. https://doi.org/10.1145/3586183.3606763

[26] Peng, B., Galley, M., He, P. et al. (2023). "Check Your Facts and Try Again: Improving LLMs with External Knowledge." *arXiv:2302.12813*.

[27] Schick, T., Dwivedi-Yu, J., Dessi, R. et al. (2023). "Toolformer: Language Models Can Teach Themselves to Use Tools." *NeurIPS 2023*, 36, 68539–68551. https://arxiv.org/abs/2302.04761

[28] Mirchandani, S., Xia, F., Florence, P. et al. (2023). "Large Language Models as General Pattern Machines." *arXiv:2307.04721*.

[29] Zhao, W. X., Zhou, K., Li, J. et al. (2023). "A Survey of Large Language Models." *arXiv:2303.18223*.

[30] Weng, L. (2023). "LLM-powered Autonomous Agents." Lilian's Blog. https://lilianweng.github.io/posts/2023-06-23-agent/

Appendix A: Negative Scenario Analysis

To validate robustness under adversarial conditions, five negative scenarios targeting system boundaries were designed:

rio	Setup	Expected	Observed
Exhaustion	All tools failure_rate=1.0	Escalate after 3 repairs; ABORTED	Correct. Terminates in <2 s. No infinite loop.
	Objective = single word	0-step plan; status=COMPLETED	Correct. Edge idx>=len(plan) triggers completion.
Skip Attempt	is_optional=False; RCA→SKIP	_skip() detects mandatory; escalates	Correct. Guard prevents skip, escalates instead.
LLM Output	LLM returns invalid _RCAOutput	Pydantic ValidationError; enter repair loop	Correct. System self-heals its own RCA failure.
	10 simultaneous POST /tasks/	Isolated per-task; no state bleed	Correct. UUID + MemorySaver ensure separation.

Table A1. Negative scenario analysis — boundary and adversarial conditions.

Appendix B: Workflow State Transition Diagram

The AgentStatus state machine follows deterministic transitions with no backward edges from COMPLETED or ABORTED, ensuring workflow finality:

```
NNING → EXECUTING → COMPLETED → LEARNING → END

                    ↓ (fail)
                    FAILED → REPAIRING → (retry) → EXECUTING
                                ↓ (max repairs)
                                ABORTED → END
```